

Programa AI-First · Fase 1

Jikkosoft — Preparación y Reto Preparado por Diego Trujillo

Principio fundacional

Todo proyecto AI empieza con `/init`.

Antes de escribir una sola spec, antes de tocar código, antes de todo: ejecuta `claude /init` en el repositorio. Esto genera el `CLAUDE.md` base y establece el contrato AI del proyecto. Sin este paso, la IA trabaja sin contexto y los resultados son impredecibles.

Estructura general

Semana	Tema	Workshops
1	Claude CLI + Spec Engineering	2
2	CLAUDE.md	2
3	Plan & Loop · MCP Servers · Subagentes · Git	2
4	Reto Fase 1 (DEV + QA en paralelo)	—

6 workshops en total · 1-2 por semana · evaluación el viernes de la semana 4.

Semana 1 — Claude CLI

Workshop 1 — Apertura + Claude CLI

Lunes

- Apertura del programa: contexto del "momento cero" y el nuevo modelo operativo AI-first.
- Instalación y configuración: Claude Code + modelo conectado a Hermes.
- **El ritual** `/init` en vivo sobre un repo vacío: qué genera, por qué importa.
- Navegación básica: slash commands, permisos, modo interactivo vs. autónomo.
- Hermes como intermediario obligatorio: toda interacción con modelos pasa por aquí.

Workshop 2 — Spec Engineering

Jueves o viernes

- Por qué la calidad de la spec supera la elección del modelo.
- La matriz real: 3 niveles de spec × 7 modelos — caso de estudio extraído del experimento interno de Diego (`hermes-exploratory`).
- Hallazgo clave: **Haiku + Spec C (88 pts) > Opus + Spec A (72 pts)**. Spec B + Sonnet es el punto de equilibrio costo/calidad.
- Ejercicio práctico: cada participante escribe 3 versiones de spec para algo de su contexto y compara outputs.

Días sin workshop: adaptación libre + punto de control (3 líneas de avance + bloqueos vía Hermes).

Semana 2 — CLAUDE.md

Workshop 3 — Anatomía del CLAUDE.md

Lunes

- El `/init` genera el esqueleto; tú lo conviertes en un contrato AI preciso.
- Componentes esenciales: authority hierarchy, safe/unsafe zones, naming conventions, model selection table.

- Revisión en vivo de un `CLAUDE.md` real de producción (21 KB, API de SILIN).
- Diferencia entre un README y un AI operating contract.

Workshop 4 — CLAUDE.md hands-on

Jueves o viernes

- Cada participante toma su repo actual (SILIN / integraciones) y escribe su propio `CLAUDE.md` desde cero.
 - **Track DEV:** orientado a construcción (safe zones para schema, API, FE).
 - **Track QA:** orientado a pruebas (qué no modificar del SUT, cómo describir el alcance de cobertura).
 - Revisión cruzada en pares.
-

Semana 3 — Plan & Loop · MCP Servers · Subagentes · Git

Workshop 5 — Plan Mode + Loop + Skills

Lunes

- `/plan` antes de cualquier tarea compleja: cómo estructurar el trabajo antes de generar código.
- Loop modes para tareas iterativas.
- Sistema `everything-claude-code`: el skill correcto para cada tipo de tarea.
 - `:tdd-workflow` → desarrollo guiado por pruebas
 - `:backend-patterns` → APIs y servicios
 - `:security-review` → revisión de seguridad
 - `:database-reviewer` → schema y queries
- Demostración completa: `/init` → `/plan` → build con skills apropiados.

Workshop 6 — MCP Servers + Subagentes + Git + Cierre

Jueves o viernes

- **MCP Servers:** Figma como fuente de diseño leíble por IA · Azure DevOps WI como PRD · `claude-mem` para memoria persistente entre sesiones.
- **Subagentes paralelos:** el patrón LLM-as-judge (un modelo evalúa el output de otro). Caso real: `job_hermes_gate` en GitLab CI.
- **Git con AI:** commits semánticos, PRs, `ai-dev-log` para trazabilidad de sesiones.
- **Cierre del curso y lanzamiento oficial de los Retos Fase 1.**

Semana 4 — Reto Fase 1

Dos tracks en paralelo

Track DEV — Construye un producto end-to-end

Misión: En 4 días, construir un producto funcional completo sin escribir código a mano. Todo se genera y se itera a través de Hermes.

Requisitos mínimos del producto:

Componente	Requisito
Backend	Servicio con lógica de negocio y API
Frontend	Interfaz funcional (web)
Base de datos	PostgreSQL, MongoDB o la de tu preferencia
Web service	Al menos un endpoint expuesto o consumido
Integración	Al menos una integración externa o entre módulos propios

Reglas:

1. Cero código manual — solo especificas, diriges, revisas e iteras.

2. Hermes es obligatorio e innegociable para toda interacción con modelos.
3. Idea libre, distinta a SILIN.
4. Repositorio público (GitHub o GitLab).

Track QA — Diseña y ejecuta una estrategia de pruebas

Misión: En 4 días, diseñar, automatizar y ejecutar una estrategia de pruebas E2E sobre la aplicación base provista (mini-tienda-base), sin escribir scripts a mano.

Requisitos mínimos de entrega:

Componente	Requisito
Plan de pruebas	Estrategia, alcance, riesgos y criterios de aceptación
Pruebas funcionales	Casos: happy path, borde y negativos
Automatización UI / E2E	Suite automatizada sobre el frontend
Pruebas de API	Contratos, estados HTTP, errores
Pruebas de datos e integración	Validación de BD y de la integración de precios
Reporte de defectos	Hallazgos con severidad, pasos de reproducción y evidencia

Reglas:

1. Cero scripts manuales — la IA los genera, tú especificas y revisas.
 2. Hermes obligatorio.
 3. No modificar el código de producción del SUT.
 4. Repositorio público (GitHub o GitLab).
-

Cronograma semana 4

Día	DEV	QA
Lunes	/init → CLAUDE.md → primera spec → inicio de construcción	/init → plan de pruebas → setup Playwright / pytest
Martes-Jueves	Backend + Frontend + DB + Integración · punto de control diario (3 líneas vía Hermes)	E2E + API + datos + escenario PRICING_FAIL=1 · punto de control diario
Viernes AM	Repo final + HERMES_CONTEXT.md	Repo final + HERMES_CONTEXT.md
Viernes PM	Demo 5-7 min + cata por grupos	Demo 5-7 min + cata por grupos

Entregables de ambos tracks

`HERMES_CONTEXT.md` (obligatorio, tan importante como el código)

Track DEV:

- Qué construiste y qué problema resuelve
- Stack y arquitectura: componentes y cómo se conectan
- Cómo usaste Hermes: skills, specs y prompts que mejor funcionaron
- Decisiones y trade-offs
- Bloqueos y cómo los resolviste
- Qué mejorarías o pedirías
- Enlace al repositorio

Track QA:

- Alcance: qué probaste y bajo qué estrategia
- Enfoque y herramientas: tipos de prueba y stack usado
- Cómo usaste Hermes: skills, specs e iteraciones

- Decisiones y trade-offs: qué priorizaste y por qué
- Bloqueos y cómo los resolviste
- Hallazgos principales / riesgos detectados
- Enlace al repositorio

Criterios de evaluación

Dimensión	Peso
Calidad del HERMES_CONTEXT.md y trazabilidad del proceso	25%
Autonomía: investigó y desbloqueó por cuenta propia	25%
Orquestación de IA: claridad de specs, iteración, manejo de contexto	18%
Funcionalidad E2E (DEV) / Cobertura y profundidad (QA)	14%
Previsión y comunicación a tiempo	10%
Criterio técnico (DEV) / Criterio de QA (QA)	8%

Se busca **criterio y adaptación**, no el proyecto más grande ni la suite más extensa.

Sobre créditos y costos

Los modelos tienen costo y las capas gratuitas se agotan. **Prever esto es parte del reto.**

Si necesitas acceso a un modelo corporativo o te vas a quedar corto de créditos, levanta la mano a tiempo — no el último día. Anticiparte y comunicar a tiempo es una de las capacidades que se están evaluando.

Dudas durante el programa: canalízalas a Diego Trujillo por Google Chat.